

of all the RDDs:

```
scala> val inputfile = sc.textFile("input.txt")
```

We apply transformations to the data by splitting each line into individual words. Earlier, one line was one entity but now each word is an entity. Next, let's count the frequency of each word and then reduce it by its key, by adding the frequency of each distinct word, using the code shown below:

```
scala> val counts = inputfile.flatMap(line => line.split(" ")).
map(word => (word, 1)).reduceByKey(_+_);
```

We can cache the output for it to persist, as follows:

```
scala> counts.cache()
```

Or we can store it to an external text file, as follows:

```
scala> counts.saveAsTextFile("output")
```

We can check the output as follows:

```
$ cd output/
$ ls -l
```

Print the output on the Spark screen, using the command shown below:

```
$ cat part-00000
$ cat part-00001
```

Analysing Big Data using the Apache Spark engine

With the advancement in technology, Web servers, machine log files, IoT, social media, user clicks, Web streaming, etc, are all generating petabytes of data, daily. Most of this is semi-structured or unstructured. This Big Data is characterised by high velocity, high volume and high variability; hence, traditional algorithms and processing technologies are unable to cope with it. MapReduce was able to process this data satisfactorily using a cluster of commodity hardware. But the ever-increasing volume of data is exceeding the capability of MapReduce due to the reasons mentioned earlier. Spark was designed as an answer to the limitations of MapReduce. It provides an abstraction of memory for sharing data and for in-memory computing. RDD can be persisted and re-used for other computations. Spark's multi-platform support, the ability to integrate with Hadoop, and its compatibility with the cloud make it tailor-made for Big Data.

In the real world, Spark is used for many applications. Banks analyse large volumes of data from sources like social media, email, complaint logs, call records, etc, to gain knowledge for credit risk assessment, customer

segmentation or targeted advertising. Even credit card fraud can be checked by it. E-commerce sites use the streaming clustering algorithm to analyse real-time transactions for advertising or to recommend products to customers by gaining insights from sources like review forums, comments, social media, etc. Shopify, Alibaba and eBay use these techniques. The healthcare sector benefits from Spark as it enables quick diagnosis and filters out individuals who are at risk. The MyFitnessPal app uses Spark to process the data of all its active users. Spark is widely used in genome sequencing and DNA analysis as millions of strands of chromosomes have to be matched. This task earlier took weeks but now takes only hours. Spark is also being used by the entertainment industry (such as Pinterest, Netflix and Yahoo News) for personalisation and recommendation systems.

Sample Big Data processing using the Apache Spark engine

Let's look at a simple application for beginners that can process Big Data. Let's take the dataset of 'Five Thirty Eight', a popular US TV show, and perform simple aggregation functions. Download the data for the past 50 years using https://github.com/fivethirtyeight/data/blob/master/daily-show-guests/daily_show_guests.csv.

Create an RDD, read the data and print the first five lines using the following code.

```
raw_data = sc.textFile("daily_show_guests.csv")
raw_data.take(5)
```

Then, split each word by using a map function, as follows:

```
daily_show = raw_data.map(lambda line: line.split(','))
daily_show.take(5)
```

Next, define a function to calculate the tally of guests each year, as shown below:

```
tally = dict()
for line in daily_show:
    year = line[0]
    if year in tally.keys():
        tally[year] = tally[year] + 1
    else:
        tally[year] = 1
```

Execute the function by using the Reduce transformation, as shown below:

```
tally = daily_show.map(lambda x: (x[0], 1))
    .reduceByKey(lambda x,y: x+y)
print(tally)
tally.take(tally.count())
```